
Tecnologie del Linguaggio Naturale

Domande e Risposte – Parte 1 (Mazzei)

Università degli Studi di Torino (UniTO)
Corso di Laurea Magistrale in Informatica
A.A. 2025/26

Alessandro Olivero

Documento generato a scopo di studio

0. Indice

1	Come funziona il Transition Based Parser (MALT)?	4
1.1	Configurazione	4
1.2	Le tre operazioni (Arc-Standard)	4
1.3	Esempio: “I booked a morning flight”	4
1.4	Oracle e classificatore	4
2	Spiegare l’uso delle HMM nel PoS Tagging	5
2.1	Componenti del modello	5
2.2	Obiettivo: decodifica	5
2.3	Algoritmo di Viterbi	5
2.4	Stima dei parametri (MLE)	5
2.5	Performance	5
3	Che differenza c’è tra MEMM e HMM?	6
3.1	MEMM – Maximum Entropy Markov Model	6
3.2	Label Bias Problem	6
4	Cosa è una Probabilistic CFG (PCFG)?	6
4.1	Probabilità di un albero sintattico	7
4.2	Utilizzo: disambiguation sintattico	7
4.3	Limitazioni	7
5	Cosa è il task dell’Aggregation (NLG)?	7
5.1	Contesto: i 7 task dell’NLG	7
5.2	Problema	7
5.3	Esempio	8
5.4	Tecniche	8
6	I 5 algoritmi per il parsing delle dipendenze sintattiche	8
6.1	Nota sulla proiettività	9
7	Il formalismo Lambda-FoL per la Semantica Computazionale	9
7.1	Principio di composizionalità (Frege)	9
7.2	Lambda astrazione e beta-riduzione	9
7.3	Grammatica semantica di base	9
7.4	Determinatori e quantificatori	10
7.5	Integrazione con la sintassi	10
8	La rappresentazione Neo-Davidsoniana	10
8.1	Semantica Davidsoniana (base)	10
8.2	Approccio Neo-Davidsoniano	10
8.3	Vantaggi	11
9	Le due ipotesi fondamentali della linguistica computazionale	11
10	La Frame-Based Semantics nei Dialogue Systems (DS)	11
10.1	Struttura di un frame	12
10.2	Algoritmo in 3 passi (pipeline NLU)	12

10.3 Gestione del dialogo (Dialogue Manager)	12
10.4 Limiti dei frame	12
11 Fare un esempio di ambiguità puramente semantica	12
11.1 Polisemia lessicale	13
11.2 Ambiguità di scope quantificazionale	13
11.3 Ambiguità del pronome (coreference)	13
11.4 Esempio classico (Mazzei)	13
12 Ambiguità sintattica: PP attachment e coordinazione	13
12.1 PP Attachment (Attacco del Sintagma Preposizionale)	13
12.2 Ambiguità della Coordinazione	14
13 Come funziona l’algoritmo CKY?	14
13.1 Pre-requisito: CNF	14
13.2 Struttura dati: tabella triangolare	14
13.3 Esempio: “Book that flight” (CKY)	15
14 Complessità dell’algoritmo CKY	15
14.1 Analisi dei loop	15
14.2 Versione probabilistica (PCKY)	15
14.3 Confronto con altri algoritmi	15
15 Dipendenze vs. Costituenti	16
15.1 Vantaggi delle dipendenze (Tesnière 1959)	16
15.2 Vantaggi dei costituenti	16
15.3 Convertibilità	16
16 La misura PARSEVAL	16
16.1 Definizione	16
16.2 Labeled vs. Unlabeled	17
16.3 Limitazioni	17
17 Cosa è un Treebank?	17
17.1 Caratteristiche	17
17.2 Esempi principali	17
17.3 Utilizzo	17
18 HMM: modello generativo o discriminativo?	18
18.1 Spiegazione	18
18.2 Confronto con i modelli discriminativi	18
19 Spiegare cosa si intende per Anatomia di un Parser	18
19.1 Relazione tra i tre componenti	19
19.2 Esempi	19
20 I 6 fenomeni notevoli del dialogo tra umani	19
21 Cosa è il PoS Tagging?	20
21.1 Classi principali	20

21.2	Ambiguità e sfide	20
21.3	Approcci	21
22	Cosa è il NER Tagging?	21
22.1	Classi tipiche (CoNLL-2003)	21
22.2	Schema di annotazione IOB2	21
22.3	Sfide	21
22.4	Approcci	21
22.5	Applicazioni	22

1. Come funziona il Transition Based Parser (MALT)?

Il **MALT Parser** (Maltparser, Nivre et al. 2008) è un parser per la sintassi a **dipendenze** che modella il parsing come una sequenza di *transizioni* su una configurazione.

Configurazione

La configurazione è una tripla (σ, β, A) :

- **Stack** σ : contiene i token già parzialmente processati (inizia con il solo ROOT).
- **Buffer** β : contiene i token ancora da processare (inizia con l'intera frase).
- **Insieme di archi** A : le relazioni di dipendenza costruite finora (inizia vuoto).

Stato iniziale: $\{[\text{root}], [w_1, \dots, w_n], \emptyset\}$

Stato finale: $\{[\text{root}], [], A_{\text{finale}}\}$

Le tre operazioni (Arc-Standard)

- **SHIFT**: sposta il primo elemento del buffer in cima allo stack.
- **LEFT-ARC**(r): crea un arco $\text{top}(\sigma) \rightarrow \text{sec}(\sigma)$ con relazione r ; rimuove $\text{sec}(\sigma)$ dallo stack. (Il top diventa testa del secondo.)
- **RIGHT-ARC**(r): crea un arco $\text{sec}(\sigma) \rightarrow \text{top}(\sigma)$ con relazione r ; rimuove il top dallo stack. (Il secondo diventa testa del top.)

Esempio: “I booked a morning flight”

Stack	Buffer	Azione	Archi
[root]	[I, booked, a, morning, flight]	SHIFT	–
[root, I]	[booked, a, morning, flight]	SHIFT	–
[root, I, booked]	[a, morning, flight]	LEFT-ARC(Subj)	booked←a
[root, booked]	[a, morning, flight]	SHIFT	–
[root, booked, a]	[morning, flight]	SHIFT	–
[root, booked, a, morning]	[flight]	SHIFT	–
[root, booked, a, morning, flight]	[]	LEFT-ARC(Mod)	flight←morning
[root, booked, a, flight]	[]	LEFT-ARC(Det)	flight←a
[root, booked, flight]	[]	RIGHT-ARC(Obj)	booked→flight
[root, booked]	[]	RIGHT-ARC(Root)	root→booked

Oracle e classificatore

In fase di **training** un *oracle* (basato su un gold treebank) indica la transizione corretta. A runtime un **classificatore** (SVM, perceptron, rete neurale) predice la

prossima azione sulla base di feature estratte dalla configurazione corrente (PoS in cima allo stack, forme, relazioni già costruite, ecc.).

La complessità è **lineare** $O(n)$: ogni token viene shiftato e rimosso esattamente una volta.

2. Spiegare l'uso delle HMM nel PoS Tagging

Un **Hidden Markov Model** (HMM) modella il PoS Tagging come un problema di *decodifica*: dato il testo osservato $w_1, \dots, w_n$, si vuole trovare la sequenza di tag $t_1, \dots, t_n$ più probabile.

Componenti del modello

- **Stati nascosti**: i tag PoS (t_i).
- **Osservazioni**: le parole (w_i).
- **Probabilità di transizione** $P(t_i \mid t_{i-1})$: quanto spesso un tag segue il precedente (stimata da corpus).
- **Probabilità di emissione** $P(w_i \mid t_i)$: quanto spesso un tag “emette” una parola.
- **Probabilità iniziale** $P(t_1)$.

Obiettivo: decodifica

$$\hat{t}_{1:n} = \arg \max_{t_{1:n}} P(t_1) \cdot \prod_{i=2}^n P(t_i \mid t_{i-1}) \cdot \prod_{i=1}^n P(w_i \mid t_i)$$

Algoritmo di Viterbi

La decodifica efficiente avviene con l'algoritmo di **Viterbi** (programmazione dinamica), che evita l'enumerazione esponenziale di tutte le sequenze. Complessità: $O(n \cdot |T|^2)$ dove $|T|$ è la dimensione del tagset.

Stima dei parametri (MLE)

$$P(t_i \mid t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}, \quad P(w_i \mid t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

Lo **smoothing** (add-one, back-off) è necessario per gestire probabilità zero su parole/transizioni non viste in training.

Performance

La baseline più semplice (*most frequent tag + unknown = noun*) raggiunge già il 92% di accuratezza. I modelli moderni (HMM, CRF, BERT fine-tuned) superano il 97% su inglese.

3. Che differenza c'è tra MEMM e HMM?

Aspetto	HMM	MEMM
Tipo di modello	Generativo	Discriminativo
Distribuzione	$P(w, t)$ congiunta	$P(t w)$ condizionale
Feature	Solo parola corrente	Feature arbitrarie sul contesto
Training	Conteggi + smoothing	Ottimizzazione (L-BFGS, SGD)
Label bias	No	Sì (problema noto)
Normalizzazione	Globale	Locale per ogni stato

MEMM – Maximum Entropy Markov Model

Il MEMM modella direttamente la distribuzione condizionale con un modello *log-linear*:

$$P(t_i | t_{i-1}, w_{1:n}) = \frac{1}{Z} \exp \left(\sum_k \lambda_k f_k(t_i, t_{i-1}, w_{1:n}, i) \right)$$

dove f_k sono feature arbitrarie (suffisso, capitalizzazione, parola precedente, parola successiva, ...) e λ_k i pesi appresi. Questo permette di usare feature molto più ricche rispetto all'HMM.

Label Bias Problem

Il principale svantaggio del MEMM è il **label bias problem**: le probabilità di transizione vengono normalizzate *localmente* per ogni stato, favorendo gli stati con pochi successori indipendentemente dall'evidenza globale. I **Conditional Random Fields** (CRF) risolvono questo problema con una normalizzazione *globale* sulla sequenza intera.

4. Cosa è una Probabilistic CFG (PCFG)?

Una **Probabilistic Context-Free Grammar** (PCFG) estende una CFG associando una probabilità a ogni regola di produzione.

Una PCFG è una quintupla $G = (N, \Sigma, R, S, P)$ dove:

- N : insieme dei simboli non-terminali.
- Σ : insieme dei simboli terminali (vocabolario).
- R : insieme delle regole di produzione $A \rightarrow \alpha$.
- $S \in N$: simbolo iniziale.
- P : funzione $P(A \rightarrow \alpha) \in [0, 1]$ con $\sum_{\alpha} P(A \rightarrow \alpha) = 1$ per ogni A .

Probabilità di un albero sintattico

La probabilità di un albero T è il *prodotto* delle probabilità di tutte le regole utilizzate:

$$P(T) = \prod_{(A \rightarrow \alpha) \in T} P(A \rightarrow \alpha)$$

Utilizzo: disambiguation sintattico

Tra tutti gli alberi $T_1, T_2, \dots$ compatibili con la grammatica per una frase data, si sceglie quello con probabilità massima: $\hat{T} = \arg \max_T P(T)$. I parametri si stimano da un treebank con MLE:

$$P(A \rightarrow \alpha) = \frac{C(A \rightarrow \alpha)}{C(A)}$$

Limitazioni

Le PCFG assumono che la probabilità di una regola sia *indipendente dal contesto* (dal non-terminale padre, dalla posizione nella frase, ecc.), producendo stime poco accurate. Le **Lexicalized PCFG** (Collins 1997, Charniak 1997) mitigano questo problema propagando informazioni lessicali attraverso l'albero.

5. Cosa è il task dell'Aggregation (NLG)?

L'**Aggregation** è il terzo dei sette task fondamentali della **Natural Language Generation** (NLG). Stabilisce *come raggruppare e combinare* le informazioni da esprimere in unità testuali coerenti e naturali.

Contesto: i 7 task dell'NLG

1. Content Determination
2. Discourse Planning
3. **Sentence Aggregation** questo task
4. Lexicalization
5. Referring Expression Generation
6. Syntactic/Morphological Realization
7. Orthographic Realization

Problema

Dato un insieme di messaggi/proposizioni da esprimere, l'aggregation decide:

- Quante frasi produrre.
- Quali proposizioni unire nella stessa frase (coordinazione, subordinazione, participiali...).

- Come evitare ripetizioni (ellissi, pronominalizzazione).

Nota: l'aggregation è il *primo* task dipendente dalla lingua specifica.

Esempio

Input (messaggi separati):

- IDENTITY(NextTrain, CaledonianExpress)
- DEPARTURETIME(CaledonianExpress, 10:00)

Senza aggregation: “The next train is the Caledonian Express. The Caledonian Express leaves at 10am.”

Con aggregation: “The next train, which leaves at 10am, is the Caledonian Express.”

Tecniche

- **Set aggregation:** raggruppamento di proposizioni con soggetto o predicato comune.
- **Conjunction:** unione con connettivi coordinanti (“e”, “ma”, “però”).
- **Embedding:** una proposizione inserita nell'altra come relativa o participiale.
- **Ellissi:** omissione di elementi recuperabili dal contesto.

Rischio: l'aggregazione può creare ambiguità, assumendo che il lettore riesca a disambiguare.

6. I 5 algoritmi per il parsing delle dipendenze sintattiche

Secondo Mazzei (lezione 6), i cinque approcci principali al dependency parsing sono:

1. Dynamic Programming

Adattamento degli algoritmi a programmazione dinamica per le dipendenze. Nel caso generale la complessità è $O(n^5)$; con l'algoritmo di **Eisner (1996)**, che sfrutta la vincolo di proiettività, scende a $O(n^3)$. Garantisce l'ottimo globale per alberi proiettivi.

2. Graph Algorithms (MST – Maximum Spanning Tree)

Il parsing è formulato come ricerca del *Maximum Spanning Tree* (o *Minimum Spanning Tree* con pesi invertiti) su un grafo completo pesato dei token. Il punteggio di ogni arco $h \rightarrow d$ è assegnato da un modello ML (*arc-factored*). L'algoritmo di **Chu-Liu/Edmonds** trova l'albero ottimale anche per strutture non proiettive. Usato in **MSTParser** (McDonald et al. 2005) – il primo parser a dipendenze di Google.

3. Constituency + Conversion

Si esegue prima il parsing a costituenti (CFG/PCFG) con algoritmi standard (CKY, Earley), poi si converte l'albero a costituenti in un albero a dipendenze tramite *head-percolation tables* (tabelle che specificano quale figlio è la testa per ogni categoria). Approccio classico, meno usato nella pratica moderna.

4. Constraint Satisfaction

Le dipendenze vengono generate da un insieme di ipotesi candidate e poi filtrate/eliminate tramite *vincoli hard* (morfosintattici, di subcategorizzazione, semantici). Usato nel parser **FIDO/TUP** (Lesmo & Torasso 1985, Torino) e in **Constraint Grammar** (Karlsson 1990).

5. Transition-Based Parsing (MALT)

Approccio greedy basato su una sequenza di transizioni (LEFT-ARC, RIGHT-ARC, SHIFT) guidata da un classificatore ML. Complessità **lineare** $O(n)$. Vedi Sezione 1 per la descrizione dettagliata.

Nota sulla proiettività

Un albero a dipendenze è **proiettivo** se ogni nodo “copre” uno span contiguo della frase (nessun arco si incrocia). Il MALT Arc-Standard produce solo alberi proiettivi; per il non-proiettivo si usa Arc-Eager o algoritmi di post-processing. MST gestisce nativamente il caso non-proiettivo.

7. Il formalismo Lambda-FoL per la Semantica Computazionale

Il **Lambda Calcolo applicato alla Logica del Primo Ordine** (*Lambda-FoL*) è il formalismo standard per costruire rappresentazioni semantiche composizionali a partire dalla struttura sintattica.

Principio di composizionalità (Frege)

Il significato di un'espressione è funzione del significato delle sue parti e del modo in cui sono combinate. Il lambda calcolo fornisce il meccanismo di combinazione.

Lambda astrazione e beta-riduzione

- **Lambda astrazione:** $\lambda x. \varphi(x)$ è una funzione che, dato x , restituisce $\varphi(x)$.
- **Beta-riduzione:** $(\lambda x. \varphi(x))(a) \Rightarrow \varphi[x \mapsto a]$

Grammatica semantica di base

Lessico:

NP: paolo $\rightsquigarrow \lambda P. P(\text{paolo})$

NP: francesca $\rightsquigarrow \lambda P. P(\text{francesca})$

V: ama $\rightsquigarrow \lambda y. \lambda x. \text{love}(x, y)$

Regole composizionali:

VP = V(NP) S = NP(VP)

Esempio – “Paolo ama Francesca”:

$$\begin{aligned} VP &= (\lambda y. \lambda x. \text{love}(x, y))(\lambda P. P(\text{francesca})) \\ &\Rightarrow \lambda x. \text{love}(x, \text{francesca}) \\ S &= (\lambda P. P(\text{paolo}))(\lambda x. \text{love}(x, \text{francesca})) \\ &\Rightarrow \text{love}(\text{paolo}, \text{francesca}) \end{aligned}$$

Determinatori e quantificatori

Per gestire “un uomo ama Francesca” si usa il *type-raising* dei determinatori:

$$\begin{aligned} \text{DT: un} &\rightsquigarrow \lambda P. \lambda Q. \exists z (P(z) \wedge Q(z)) \\ \text{DT: ogni} &\rightsquigarrow \lambda P. \lambda Q. \forall z (P(z) \Rightarrow Q(z)) \\ \text{DT: nessun} &\rightsquigarrow \lambda P. \lambda Q. \forall z (P(z) \Rightarrow \neg Q(z)) \end{aligned}$$

Esempio – “Un uomo ama Francesca”:

$$(\lambda P. \lambda Q. \exists z (P(z) \wedge Q(z)))(\lambda z. \text{man}(z)) \Rightarrow \lambda Q. \exists z (\text{man}(z) \wedge Q(z))$$

Applicato al VP $\lambda x. \text{love}(x, \text{francesca})$:

$$\exists z (\text{man}(z) \wedge \text{love}(z, \text{francesca}))$$

Integrazione con la sintassi

Il formalismo si integra con grammatiche categoriali (CCG) o con regole composizionali bottom-up su alberi sintattici. A ciascun lessema si assegna un’*entrata semantica* in Lambda-FoL; il significato dell’intera frase si ottiene componendo dal basso verso l’alto.

8. La rappresentazione Neo-Davidsoniana

La rappresentazione **Neo-Davidsoniana** (Parsons 1990) estende la semantica Davidsoniana classica per gestire in modo uniforme argomenti verbali obbligatori e modificatori facoltativi.

Semantica Davidsoniana (base)

Davidson (1967) propose di introdurre una **variabile evento** e come argomento implicito del verbo, permettendo di trattare gli avverbi come predicati sull’evento:

$$\text{“Mario corre velocemente”} \rightsquigarrow \exists e [\text{correre}(e, \text{Mario}) \wedge \text{veloce}(e)]$$

Approccio Neo-Davidsoniano

Il Neo-Davidsonismo decompone ulteriormente: **tutti** gli argomenti tematici diventano predicati binari sull’evento, separando completamente la struttura argomentale:

“Mario mangia una mela” $\rightsquigarrow \exists e [mangiare(e) \wedge \theta_{ag}(e, \text{Mario}) \wedge \theta_{th}(e, \text{mela})]$

dove θ_{ag} = ruolo *agente*, θ_{th} = ruolo *tema*.

Vantaggi

- **Trattamento uniforme** di argomenti obbligatori e modificatori opzionali: gli avverbi diventano semplici predicati sull’evento come gli argomenti.
- **Inferenza**: da “Mario mangia una mela velocemente” si ottiene “Mario mangia una mela” semplicemente per eliminazione del congiunto *veloce*(*e*).
- Integrazione naturale con risorse come **VerbNet** e **PropBank**, che usano ruoli tematici espliciti.
- Permette di aggregare informazioni provenienti da frasi diverse sullo stesso evento.

9. Le due ipotesi fondamentali della linguistica computazionale

I. Ipotesi di regolarità (*ipotesi linguistica*):

Il linguaggio naturale è strutturato e governato da *regolarità sistematiche* – fonologiche, morfologiche, sintattiche, semantiche – che possono essere descritte formalmente. Questa ipotesi giustifica l’uso di grammatiche formali, automi, regole.

II. Ipotesi di corpora (*ipotesi statistica/empirica*):

I dati linguistici reali (corpora) contengono informazione sufficiente per *apprendere, stimare e valutare* modelli del linguaggio. Il comportamento linguistico è apprendibile dall’osservazione di grandi quantità di testo.

Queste due ipotesi giustificano rispettivamente:

- L’approccio **simbolico/rule-based**: grammatiche, lessici, parser formali.
- L’approccio **statistico/neurale**: n-gram, HMM, reti neurali, Transformers.

Nella linguistica computazionale moderna i due approcci si **integrano** (es. parser neuro-simbolici, PCFG, CRF).

10. La Frame-Based Semantics nei Dialogue Systems (DS)

La **Frame-Based Semantics** è l’approccio dominante nella comprensione del linguaggio naturale (NLU) nei sistemi di dialogo orientati al task. Si basa sul concetto di **frame**: una struttura dati predefinita che rappresenta un’intenzione utente e i parametri necessari per soddisfarla.

Struttura di un frame

- **Intent (intenzione)**: l'azione che l'utente vuole compiere (es. `prenota_volo`).
- **Slot**: i campi parametrici da riempire (es. `ORIGINE`, `DESTINAZIONE`, `DATA`).
- **Tipo**: il tipo di valore atteso per ogni slot (città, data, orario, ...).
- **Valore**: il valore effettivo estratto dall'enunciato utente.

Esempio – frame `prenota_volo` (GUS, 1977):

Slot	Tipo	Domanda generata
ORIGINE	città	“Da quale città parti?”
DESTINAZIONE	città	“Dove vuoi andare?”
DATA_PARTENZA	data	“In che giorno vuoi partire?”
ORA_PARTENZA	orario	“A che ora preferisci partire?”
COMPAGNIA	compagnia	“Hai una compagnia preferita?”

Algoritmo in 3 passi (pipeline NLU)

1. **Domain Detection**: classificare l'enunciato nel dominio corretto (es. voli vs. hotel vs. meteo). Produce un insieme di frame candidati.
2. **Intent Detection**: selezionare il frame più probabile (es. `prenota` vs. `cancella` vs. `cerca`).
3. **Slot Filling**: estrarre i valori degli slot tramite NER e classificatori dedicati.

Gestione del dialogo (Dialogue Manager)

L'utente può riempire più slot in un unico enunciato:

“Voglio un volo da SF a Denver one-way, parto dopo le 17 di martedì”

→ `ORIGINE=SF`, `DESTINAZIONE=Denver`, `TIPO=one-way`, `ORA_MIN=17:00`, `DATA=martedì`

Quando tutti gli slot obbligatori sono riempiti il sistema esegue la query al database. Per gli slot mancanti, il sistema genera domande specifiche tramite **NLG**.

Limiti dei frame

I frame semplici non gestiscono bene task complessi con struttura ricorsiva (es. prenotare sia il volo *che* l'hotel). Soluzioni: gerarchie di frame, agenda di task.

11. Fare un esempio di ambiguità puramente semantica

L'ambiguità **puramente semantica** si manifesta quando una frase ha una struttura sintattica *unica* ma consente più interpretazioni semantiche.

Polisemia lessicale

“Ho visto **la banca**.”

→ Interpretazione 1: istituto finanziario.

→ Interpretazione 2: riva di un fiume.

La struttura sintattica (NP oggetto diretto) è identica in entrambi i casi; l'ambiguità è puramente nel significato del lessema “banca”.

Ambiguità di scope quantificazionale

“Ogni studente ha letto un libro.”

→ *Lettura 1* ($\forall > \exists$): $\forall x [stud(x) \Rightarrow \exists y [libro(y) \wedge legge(x, y)]]$

(ogni studente ha letto un libro qualsiasi, possibilmente diverso)

→ *Lettura 2* ($\exists > \forall$): $\exists y [libro(y) \wedge \forall x [stud(x) \Rightarrow legge(x, y)]]$

(c'è uno specifico libro che tutti gli studenti hanno letto)

Ambiguità del pronome (coreference)

“Mario ha detto a Luca che **lui** aveva ragione.”

→ “lui” = Mario *oppure* “lui” = Luca.

Esempio classico (Mazzei)

“I made her duck.”

- **Ambiguità morfologica:** *duck* = sostantivo (anatra) o verbo (abbassarsi).
- **Ambiguità sintattica:** *make* = causativo (→ “le ho fatto abbassare la testa”) o creativo (→ “le ho cucinato l'anatra”).
- **Ambiguità semantica:** *make* ha più significati.

12. Ambiguità sintattica: PP attachment e coordinazione

PP Attachment (Attacco del Sintagma Preposizionale)

Il **PP attachment** è uno dei fenomeni di ambiguità sintattica più studiati: un sintagma preposizionale (PP) può essere *attaccato* a costituenti diversi della frase, producendo interpretazioni diverse.

Esempio canonico – “I saw the man with the telescope.”

- **Attacco al VP** *saw*: ho usato il telescopio per vedere l'uomo.
[VP [V saw] [NP the man] [PP with the telescope]]
- **Attacco all'NP** *the man*: ho visto l'uomo che aveva il telescopio.
[VP [V saw] [NP the man [PP with the telescope]]]

Esempio italiano – “Ho mangiato la torta con la forchetta”:

- PP al VP: la forchetta è lo *strumento* usato per mangiare.
- PP all'NP: la torta *aveva* la forchetta (es. di accompagnamento).

La risoluzione è un problema *statistico e semantico*: richiede conoscenza delle restrizioni di selezione del verbo e delle co-occorrenze lessicali.

Ambiguità della Coordinazione

La coordinazione con “e” (o *and*) può legarsi a costituenti di livello diverso.

Esempio – “Ragazze e ragazzi intelligenti”:

- Lettura 1: [ragazze] e [ragazzi intelligenti] → solo i ragazzi sono intelligenti.
- Lettura 2: [ragazze intelligenti] e [ragazzi intelligenti] → entrambi intelligenti.

Esempio – “old men and women”:

- [[old men] and [women]] → solo gli uomini sono vecchi.
- [old [[men] and [women]]] → sia uomini che donne sono vecchi.

L’ambiguità della coordinazione si intreccia spesso con il PP attachment, producendo strutture con un numero esponenziale di analisi possibili.

13. Come funziona l’algoritmo CKY?

Il **Cocke-Kasami-Younger (CKY)** è un algoritmo di parsing a *chart* basato su programmazione dinamica per grammatiche in **Forma Normale di Chomsky (CNF)**.

Pre-requisito: CNF

In CNF ogni regola ha una delle due forme: $A \rightarrow BC$ (due non-terminali) oppure $A \rightarrow w$ (un terminale). Qualsiasi CFG può essere convertita in CNF eliminando ε -produzioni, produzioni unitarie ($A \rightarrow B$) e binarizzando le produzioni con più di 2 simboli.

Struttura dati: tabella triangolare

Si costruisce una tabella $T[i][j]$ (con $0 \leq i < j \leq n$) che contiene l’insieme dei non-terminali capaci di derivare la sottostringa $w_{i+1} \cdots w_j$.

Inizializzazione (span di lunghezza 1, $j = i + 1$):

$$T[i][i+1] = \{A \mid A \rightarrow w_{i+1} \in R\}$$

Ricorsione (lunghezza $l = 2, \dots, n$; per ogni $i, j = i + l$):

$$T[i][j] = \bigcup_{k=i+1}^{j-1} \{A \mid A \rightarrow BC \in R, B \in T[i][k], C \in T[k][j]\}$$

Riconoscimento: la frase è grammaticale sse $S \in T[0][n]$.

Esempio: “Book that flight” (CKY)

Regole di esempio: $S \rightarrow \text{NP VP}$, $\text{NP} \rightarrow \text{DET NOM}$, $\text{NOM} \rightarrow \text{N}$, $\text{VP} \rightarrow \text{V NP}$, $V \rightarrow \text{book}$, $\text{DET} \rightarrow \text{that}$, $N \rightarrow \text{flight}$.

	[0, 1]	[1, 2]	[2, 3]
[0, 1]	V, NP	–	–
[1, 2]	–	DET	–
[2, 3]	–	–	N, NOM
[0, 2]	–	–	–
[1, 3]	–	NP ($\leftarrow$ DET + NOM)	
[0, 3]	VP ($\leftarrow$ V + NP), S ($\leftarrow$ NP + VP)		

$S \in T[0][3]$: la frase è riconosciuta.

14. Complessità dell’algoritmo CKY

Complessità temporale: $O(n^3 \cdot |G|)$

dove n è la lunghezza dell’input e $|G|$ è la dimensione della grammatica (numero di regole binarie).

Complessità spaziale: $O(n^2 \cdot |N|)$

dove $|N|$ è il numero di non-terminali.

Analisi dei loop

I tre loop annidati scorrono:

1. Lunghezza dello span l : n valori.
2. Posizione di inizio i : al massimo n valori.
3. Punto di split k : al massimo n valori.

Per ogni tripla (i, j, k) si verificano tutte le regole $A \rightarrow BC$: $O(|G|)$ operazioni.

Versione probabilistica (PCKY)

Con grammatiche PCFG, si memorizza anche la probabilità massima di ogni span: complessità $O(n^3 \cdot |G|)$ invariata, ma con un fattore costante maggiore per il calcolo dei prodotti di probabilità.

Confronto con altri algoritmi

Algoritmo	Caso peggiore	Note
CKY	$O(n^3 \cdot G)$	Solo CNF
Earley	$O(n^3)$	CFG generali; $O(n^2)$ per grammatiche non ambigue
MALT (dep.)	$O(n)$	Greedy, non garantisce ottimo globale
MST (dep.)	$O(n^2)$	Ottimo globale con CL/Edmonds

15. Dipendenze vs. Costituenti

Aspetto	Costituenti	Dipendenze
Struttura	Albero con nodi interni (NP, VP...)	Albero con archi testa→dipendente
Nodi	Terminali + non-terminali	Solo terminali (le parole)
Head	Non esplicita (serve lessicalizzazione)	Esplicita per ogni arco
Ordine	Fortemente dipendente dall'ordine	Robusto a variazioni d'ordine
Relazioni	Struttura sintagmatica	Relazioni semantico-funzionali
Risorse	Penn Treebank (PTB), TIGER	Universal Dependencies (UD)
Parser	CKY, Earley, chart parser	MALT, MST, biaffine
Formalismo	CFG, PCFG, TAG	È il formalismo stesso

Vantaggi delle dipendenze (Tesnière 1959)

- Generalizzazione cross-linguistica: le relazioni testa-dipendente sono universali.
- Identificazione diretta delle relazioni predicato-argomento (superiore alla CFG per la semantica).
- Semplicità e trasparenza della struttura.
- Adatto a lingue a ordine libero (es. ceco, finlandese, latino).

Vantaggi dei costituenti

- Modellazione esplicita della struttura frasale.
- Integrazione naturale con grammatiche formali (CFG, CCG).
- Ricca tradizione e risorse (PTB con 1M parole annotate).

Convertibilità

Le due rappresentazioni sono parzialmente intercambiabili: le *head-percolation tables* permettono di estrarre dipendenze da un albero a costituenti, e viceversa. Le **Universal Dependencies** (UD) sono oggi lo standard multilinguistico per le dipendenze (100+ lingue).

16. La misura PARSEVAL

PARSEVAL è la metrica standard per valutare i parser a *costituenti* (Black et al. 1991).

Definizione

Dati l'albero prodotto dal parser (*proposed*) e l'albero di riferimento (*gold*), un **costituente** è identificato da una tripla (etichetta, inizio, fine). Si contano i costituenti proposti che coincidono con quelli gold.

$$\text{Precision} = \frac{|\text{proposti corretti}|}{|\text{proposti}|} \quad \text{Recall} = \frac{|\text{proposti corretti}|}{|\text{gold}|} \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

Labeled vs. Unlabeled

- **Labeled PARSEVAL (LP/LR)**: la tripla include l'etichetta del non-terminale (es. NP, VP).
- **Unlabeled PARSEVAL (UP/UR)**: conta solo la posizione dello span, ignorando l'etichetta.

Limitazioni

PARSEVAL non penalizza uniformemente tutti gli errori; è sensibile alle scelte di annotazione del treebank. Per la sintassi a dipendenze si usano:

- **UAS** (Unlabeled Attachment Score): % archi con testa corretta.
- **LAS** (Labeled Attachment Score): % archi con testa e relazione corretti.

17. Cosa è un Treebank?

Un **treebank** è un corpus di testo annotato con struttura sintattica (alberi), creato manualmente da linguisti esperti o semi-automaticamente (parser + revisione umana).

Caratteristiche

- Ogni frase è associata a un albero sintattico (a costituenti o a dipendenze).
- Le annotazioni includono spesso anche PoS tag e informazioni morfologiche.
- La creazione è costosa: nelle prime versioni, 1–2 ore per frase.

Esempi principali

Treebank	Lingua	Tipo	Dimensione
Penn Treebank (PTB)	Inglese	Costituenti	~1M token
Universal Dependencies (UD)	100+ lingue	Dipendenze	>200 treebank
Turin University Treebank (TUT)	Italiano	Dipendenze	~100K token
Prague Dependency Treebank	Ceco	Dipendenze	~1.5M token

Utilizzo

I treebank sono fondamentali per:

1. **Training** di parser supervisionati (MALT, biaffine, ...).
2. **Stima** dei parametri di PCFG via MLE.

3. **Valutazione** (gold standard per PARSEVAL, UAS/LAS).
4. **Ricerca linguistica** cross-linguistica (tramite UD).

18. HMM: modello generativo o discriminativo?

HMM è un modello *generativo*.

Spiegazione

Un modello generativo modella la **distribuzione congiunta** $P(X, Y)$ di osservazioni e etichette. Dall'HMM si può *generare* (campionare) una coppia (sequenza di tag, sequenza di parole) secondo il processo:

$$P(w_{1:n}, t_{1:n}) = P(t_1) \prod_{i=2}^n P(t_i | t_{i-1}) \prod_{i=1}^n P(w_i | t_i)$$

La classificazione avviene poi per inferenza bayesiana: $\hat{t} = \arg \max_t P(t | w) = \arg \max_t P(w, t)$.

Confronto con i modelli discriminativi

Modello	Tipo	Distribuzione modellata
HMM	Generativo	$P(X, Y)$
Naive Bayes	Generativo	$P(X, Y)$
MEMM	Discriminativo	$P(Y X)$
CRF	Discriminativo	$P(Y X)$ globale
SVM	Discriminativo	decision boundary

I modelli discriminativi modellano direttamente $P(Y | X)$, permettendo di usare feature arbitrarie; generalmente più accurati in classificazione. I generativi possono generare dati sintetici e gestire meglio i missing data.

19. Spiegare cosa si intende per Anatomia di un Parser

Con “**anatomia di un parser**” (Steedman 2000) si intende la scomposizione di qualsiasi sistema di parsing in tre componenti fondamentali:

1. Grammatica (Grammar / Model)

Specifica le strutture sintattiche ammissibili e i loro pesi. Può essere:

- Context-Free Grammar (CFG), PCFG, TAG, CCG, Grammatica a dipendenze.

2. Algoritmo (Algorithm)

La procedura di ricerca nello spazio delle strutture. Si articola in due dimensioni:

- *Strategia di ricerca*: top-down, bottom-up, left-to-right, beam search.
- *Organizzazione della memoria*: backtracking (DFS), programmazione dinamica (chart), greedy.

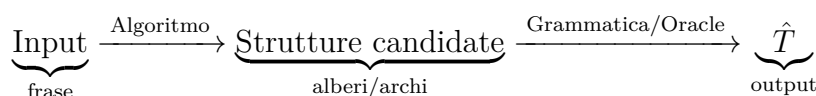
Esempi: CKY (bottom-up, DP), Earley (top-down, DP), MALT (greedy, nessuna DP).

3. Oracolo (Oracle)

Fornisce i “target” corretti durante il training. Può essere:

- *Rule-based*: determina univocamente la mossa corretta.
- *Probabilistico*: classifica le mosse tramite un modello ML addestrato su treebank.

Relazione tra i tre componenti



Esempi

Parser	Grammatica	Algoritmo	Oracolo
CKY	CFG/PCFG	Bottom-up, DP	MLE su treebank
MALT	Dipendenze	Greedy, transition	Classificatore ML
MSTParser	Dipendenze	CL/Edmonds	Arc-factored ML
Biaffine	Dipendenze	MST neurale	BiLSTM+biaffine

20. I 6 fenomeni notevoli del dialogo tra umani

Secondo Mazzei (lezione 10), i sei fenomeni fondamentali che caratterizzano il dialogo umano sono:

1. Turns (Turni)

Il dialogo è organizzato in *turni*: ogni contributo di un partecipante è un turno. Il **turn-taking** regola quando prendere, tenere o cedere la parola, ed è segnalato da cue prosodici, sintattici e gestuali. Sfide tecniche: gestione dell'interruzione (*barge-in*), rilevamento della fine del turno (*end-pointing*), pause interne al turno.

2. Speech Acts (Atti linguistici, Austin & Searle)

Ogni enunciato compie un'*azione* oltre a trasportare contenuto proposizionale. Quattro classi principali:

- **Constatives**: asseriscono qualcosa (answering, claiming, denying, stating).
- **Directives**: richiedono azioni all'interlocutore (asking, ordering, requesting, advising).
- **Commissives**: impegnano il parlante a un'azione futura (promising, planning, betting).
- **Acknowledgments**: esprimono atteggiamenti sociali (greeting, thanking, apologizing).

3. Grounding (Mutua comprensione)

I partecipanti costruiscono attivamente un *Common Ground*: si assicurano che il messaggio sia stato compreso tramite conferme esplicite (“ok”, “capisco”) o implicite (continuare il dialogo senza correzioni). Principio di **closure** (Clark 1996): i partecipanti hanno bisogno di evidenza che l’azione comunicativa abbia avuto successo.

4. Dialogue Structure (Struttura del dialogo)

Il dialogo ha una struttura locale e globale:

- **Adjacency pairs**: coppie di atti adiacenti (domanda-risposta, proposta-accettazione/rifiuto, saluto-controsaluto).
- **Sub-dialogues**: sotto-dialoghi per chiarimento o correzione, poi ritorno alla sequenza principale.
- **Presequences**: sequenze preparatorie prima della richiesta principale.

5. Conversational Initiative (Iniziativa conversazionale)

In alcuni dialoghi il controllo è in mano a uno solo (*system initiative*: il sistema fa domande, l’utente risponde). Nella maggior parte dei dialoghi umani l’iniziativa è **mista** (*mixed initiative*): ognuno può guidare la conversazione a turno. Gestire la mixed initiative è uno dei problemi più difficili per i dialogue systems.

6. Implicature (Grice 1975)

I parlanti comunicano più di quanto dicano letteralmente, sfruttando le **massime conversazionali**: *quantità* (di’ abbastanza, non di’ troppo), *qualità* (di’ il vero), *relazione* (sii pertinente), *modo* (sii chiaro). Le *implicature conversazionali* sono inferenze pragmatiche: “Sai che ore sono?” implica una richiesta di dire l’ora, non un’interrogazione sulle capacità del destinatario.

21. Cosa è il PoS Tagging?

Il **Part-of-Speech (PoS) Tagging** è il task di assegnare a ciascun token di una frase la sua **categoria grammaticale** (parte del discorso), disambiguando i token ambigui in base al contesto.

Classi principali

- **Classi aperte** (crescono con la lingua): sostantivi (NOUN), verbi (VERB), aggettivi (ADJ), avverbi (ADV).
- **Classi chiuse** (insieme finito): determinativi (DET), preposizioni (ADP), pronomi (PRON), congiunzioni (CONJ), punteggiatura (PUNCT), ecc.

Esempio (Universal PoS, UD):

Il/DET gatto/NOUN nero/ADJ dorme/VERB sul/ADP tetto/NOUN ./PUNCT

Ambiguità e sfide

- Circa l’85% dei *tipi* lessicali è non ambiguo, ma il ~ 60% dei *token* nel testo è ambiguo (es. “back”: ADJ / NOUN / VERB / ADV / PART in inglese).

- **Parole sconosciute** (OOV): gestite con feature morfologiche (prefissi, suffissi, capitalizzazione).

Approcci

- **Rule-based** (ENGTWOL): analizzatore morfologico + regole di eliminazione delle ambiguità.
- **Statistici**: HMM, MEMM, CRF (con feature morfologiche e di contesto).
- **Neurali**: BiLSTM+CRF, Transformer (BERT fine-tuned) – stato dell’arte > 97% su inglese.

Baseline semplice: assegnare a ogni parola il tag più frequente nel training + “unknown → NOUN” → già ~ 92% di accuratezza.

22. Cosa è il NER Tagging?

Il **Named Entity Recognition (NER)** è il task di identificare ed etichettare le *entità nominate* nel testo: nomi di persone, luoghi, organizzazioni, date, quantità, prodotti, ecc.

Classi tipiche (CoNLL-2003)

Etichetta	Significato
PER	Persona
ORG	Organizzazione
LOC	Luogo geografico
MISC	Altro (nazionalità, eventi, prodotti)

Schema di annotazione IOB2

Ogni token riceve un tag nel formato B-TYPE (Begin – inizio entità), I-TYPE (Inside – interno entità), O (Outside – non entità):

Mario/B-PER Rossi/I-PER lavora/O per/O Fiat/B-ORG a/O
Torino/B-LOC ./O

Sfide

- **Entità multi-token** (span): “New York City”, “Università degli Studi di Torino”.
- **Ambiguità**: “Apple” = azienda o frutto? Dipende dal contesto.
- **Nested entities**: “[Università di [Torino_{LOC}]_{ORG}]”.

Approcci

- **Rule-based**: gazetteers (liste di nomi noti) + pattern regex.
- **Statistici**: CRF con feature morfologiche, di capitalizzazione e di contesto.

- **Neurali:** BiLSTM-CRF, BERT fine-tuned – F1 $\approx$ 93% su CoNLL-2003 EN.

Applicazioni

NER è componente fondamentale di: Information Extraction, Question Answering, Knowledge Base Population, Relation Extraction, Machine Translation, sistemi biomedici (cTAKES, MetaMap).